Project Interim Report

On

Music Playlist Management System

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Student Name - Rohit Kerketta
Reg No. RU-25-11199

Bachelor of Technology in Computer Science & Engineering in association with Google

Under the Guidance of

Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The Music Playlist Management System is a specialized software application designed to help users systematically organize and interact with their digital music collections in a structured environment. In an era where listeners have access to millions of songs, an automated system is essential to prevent digital clutter and allow users to create specific sets of music for different moods or activities. This project provides a robust framework to store essential song information, such as the title and the artist, and group them into a single, manageable playlist using a Song class and a Playlist class.

The system goes beyond just listing tracks; it allows users to dynamically add new songs to their collection while ensuring the storage capacity is monitored. One of its most distinctive features is the Shuffle function, which utilizes randomization logic like the Fisher-Yates algorithm to rearrange the songs, ensuring a fresh and unpredictable listening experience every time. By utilizing Object-Oriented Programming, the project demonstrates how complex collections of data can be managed through aggregation and encapsulation. Ultimately, this project serves as a foundational tool that models real-world playback behavior, making it easier for users to maintain a seamless and organized audio queue.

Introduction

Digital music players rely on efficient management to play songs seamlessly. This project focuses on building that foundation using a building block approach. We create a blueprint for a single Song, and then a larger blueprint for a Playlist that can hold many of those songs. This mimics the real-world HAS-A relationship where a Playlist has a collection of songs.

Key Features:

- The System securely keeps track of song names and artists who performed them.
- It allows the user to see exactly what is playing next in a clean numbered list.
- It includes a shuffle feature so the order of music is not predictable.

Objectives:

- Effectively store and manage song details without manual paperwork.
- Use code to instantly rearrange songs at the click of a button.
- Practice using classes and aggregation by putting objects inside other objects.
- Use modern coding loops to quickly walk through the list and display it to the user.

Scope

This project is a perfect starting point for a basic media player. Currently, it works within the computer's temporary memory by using arrays to store objects. This makes it suitable for small personal collections. In the future, this can be extended by connecting it to a database so your songs are saved even after adding a Graphic User Interface with buttons and album art to make it look like a professional app.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code/Eclipse/Edit Plus)

Methodology

To understand how this project works, imagine you are organizing a physical folder of song lyrics. Here is the step by step process:

1. Creating the Blueprint

   First, we define what a song is In our code, every song must have two pieces of information, which are a title and an artist.

2. Preparing the Container

   We then create a playlist box. Inside this box, we set aside a specific amount of space called an array to hold our song objects.

3. Adding Music

   When you want to add a song, the system checks if there is still room in the box. If there is space, it places the song in the next available slot and updates the total count.

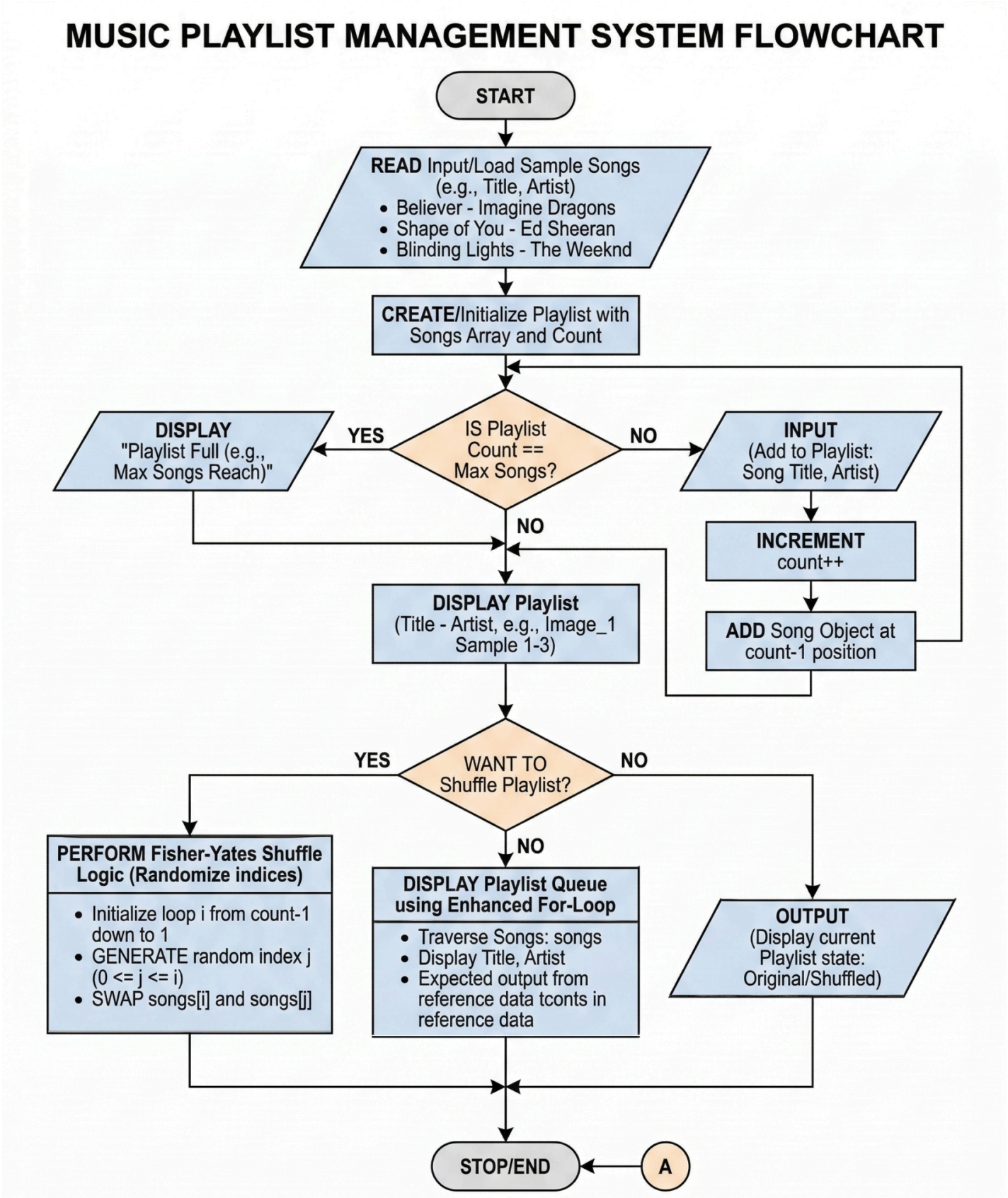
4. Mixing it Up

   To shuffle, the system looks at all the songs in the slots and starts swapping them around randomly. This is similar to shuffling a deck of cards until the original order is completely changed.

5. Reading the list

   Finally, the system goes through the slots one by one and prints the names of the songs and artists on your screen so you can see your updated playlist.

Flowchart


MUSIC PLAYLIST MANAGEMENT SYSTEM FLOWCHART
START
READ Input/Load Sample Songs (e.g., Title, Artist)
• Believer - Imagine Dragons
• Shape of You - Ed Sheeran
• Blinding Lights - The Weeknd
CREATE/Initialize Playlist with Songs Array and Count
IS Playlist Count == Max Songs?
YES
DISPLAY "Playlist Full (e.g., Max Songs Reach)"
NO
INPUT (Add to Playlist: Song Title, Artist)
INCREMENT count++
ADD Song Object at count-1 position
NO
DISPLAY Playlist (Title - Artist, e.g., Image_1 Sample 1-3)
WANT TO Shuffle Playlist?
YES
PERFORM Fisher-Yates Shuffle Logic (Randomize indices)
• Initialize loop i from count-1 down to 1
• GENERATE random index j (0 <= j <= i)
• SWAP songs[i] and songs[j]
NO
DISPLAY Playlist Queue using Enhanced For-Loop
• Traverse Songs: songs
• Display Title, Artist
• Expected output from reference data tconts in reference data
NO
OUTPUT (Display current Playlist state: Original/Shuffled)
STOP/END
A

Implementation

In this project, the system is structured into three primary classes that work together to manage a digital music collection using Object-Oriented Programming (OOP) principles.

1. Song Class

- **Purpose**: This acts as a blueprint for individual music tracks.
- **Logic**: It encapsulates the data for a song, specifically the **Title** and **Artist**. It uses a constructor to initialize these values and provides "getter" methods to retrieve the song details without allowing direct modification from outside the class, demonstrating **Encapsulation**.

2. Playlist Class

- **Purpose**: This class manages a collection of Song objects.
- **Logic**:
  - ★ It maintains an **array** of Song objects to store the data.
  - ★ It uses a count variable to keep track of how many songs have been added.
  - ★ The primary logic involves adding new song objects into the next available index of the array, ensuring the playlist does not exceed its predefined size.

3. Music (Main) Class

- **Purpose**: This is the entry point of the program.
- **Logic**:
  - ☐ It handles the **User Interface** via the terminal.
  - ☐ It uses a Scanner object to capture user input.
  - ☐ The logic follows a menu-driven approach where the user can choose to add songs, view the playlist, or exit. It coordinates the interaction between the user and the Playlist class.

Gantt Chart

Music Playlist Management System: Combined Design & Planning Table

Phase / Component	Description	Logic & OOP Concepts
System Design	Define Song and Playlist structures and relationships.	Establishes **Object Aggregation (HAS-A)**.
Song Class	Blueprint for individual music tracks.	**Encapsulation**: Uses private attributes and getters.
Playlist Class	Manages the collection of song objects.	Handles **Arrays of Objects** for data storage.
addSong() Method	Logic to add songs to the array.	**Decision Logic**: Checks if the array is full before adding.
shufflePlaylist()	Randomly rearranges the array elements.	**Randomization**: Implements Fisher-Yates shuffle logic.
Display Queue	Shows current song order.	Uses **Enhanced for-loop traversal** (`for (Song s : songs)`).
Main (Music) Class	Entry point and coordinator.	Implements **Modular Design** and handles terminal UI.

Conclusion

This project shows how to build a basic music player system by organizing code into logical "objects". By creating a **Song** class to hold track details and a **Playlist** class to manage them, I was able to model how real digital music players function.

Throughout this process, I successfully achieved several goals:

- **Organization**: I used a "blueprint" (Song class) to keep song names and artists neatly packaged together.
- **Storage**: I built a playlist that can hold a specific number of songs using a list-like structure (an array).

- **Smart Logic**: I added a "shuffle" feature that moves songs around randomly, just like a "mix" mode on a phone.
- **Simple Display**: I made it easy to view the music queue by having the program go through the list one by one and print the details.

In short, this project helped me understand how to take a real-world idea—like a music playlist—and turn it into a working computer program that is easy to read and use.

References

[1] Book Reference[1] H. Schildt, Java: The Complete Reference, 12th ed. New York: McGraw-Hill Education, 2021. (Used for implementing Java class structures, constructor logic, and array handling.)

[2] Website Reference
[2] Oracle, "The Java™ Tutorials: Arrays," Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html
(Used for logic regarding maintaining the Song[] array and tracking object counts.)

[3] Website Reference
[3] GeeksforGeeks, "Fisher-Yates Shuffle Algorithm," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/shuffle-a-given-array-using-fisher-yates-shuffle-algorithm/
(Used to implement the randomization logic for the shufflePlaylist() method.)

[4] Website Reference[
4] Oracle, "The Java™ Tutorials: The Enhanced for Loop," Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/for.html